



**BAI22123 Object-Oriented Programming
Project Description
September 2026 Semester**

Lecturer	: Dr. Ng Choon Ching
Email	: ngchoonching@raffles-university.edu.my
Due Date	: 20 November 2026 (Week 12), 5:00 PM
Marks	: 100 (Weighting 50%)
Total SLT	: 20 Hours

Course Learning Outcomes :

At the end of this module, students are expected to be able to:

1. Describe the benefits of object-oriented design and explain when it is an appropriate methodology to use.
2. Exhibit effective implementation of object-oriented concepts including inheritance, polymorphism, encapsulation, and abstraction.
3. Apply object-oriented programming concepts to solve real-world programming challenges.

1. Project Overview

The objective of this project is to give students the opportunity to apply the C++ programming and Object-Oriented Programming (OOP) concepts learned throughout the course to the development of a functional software application.

Unlike a conventional group project built to a fixed specification, this project allows each student to independently select and develop a project based on their own interests.

Students may develop a:

- Management system
- Simulation
- Utility application
- Information management system
- Business application
- Productivity application
- Data-processing application
- Other suitable C++ application

The selected project must demonstrate appropriate use of basic C++ programming syntax, object-oriented programming concepts, and, where appropriate, advanced C++ features.

Each student is responsible for designing, implementing, testing, documenting and presenting their own project, and must additionally pass an individual oral presentation that verifies their understanding of the code and OOP concepts they used.

Students are required to work with a partner for peer evaluation and technical review. The partner does not develop the project with the student — each student develops and submits their own independent project.

Item	Detail
Project Type	Individual project (independent development)
Development Language	C++
Recommended Weighting	50% of Course Grade
Total Marks	100 (+ up to 10 bonus marks for advanced features)
Mark Split	50% Project Development, Documentation & Demonstration + 50% Oral Presentation
Peer Component	Partner-based technical review (assessed within the 50% development component)
Project Theme	Student-selected application

2. Project Objectives

The project aims to enable students to:

1. Apply fundamental C++ programming syntax and constructs.
2. Design software using classes and objects.
3. Apply object-oriented programming principles appropriately.
4. Demonstrate encapsulation, abstraction, inheritance and polymorphism.
5. Apply appropriate data structures and algorithms.
6. Develop a functional C++ application.
7. Test and debug the developed application.
8. Explain and justify their programming and design decisions under oral questioning.
9. Explore advanced C++ features beyond the basic course requirements.

3. Individual Project Requirement

This is an individual project. Each student must independently: select a project topic; define the project requirements; design the system and classes; produce UML diagrams; write, test and document the C++ program; and demonstrate the program.

Students may discuss ideas and provide general assistance to their peers. However, the submitted source code, design and implementation must be the student's own work — and each student must be able to defend that work unaided during the oral presentation.

Students will be paired with another student for peer evaluation. The project is individual development with partner-based peer evaluation — it is not group development with a group submission. Two students may develop similar types of applications, but each must have their own implementation and design.

4. Project Topic Selection

Students are free to choose their own project. Examples include:

Management Systems

- Library Management System
- Restaurant Management System
- Hotel Management System
- Inventory Management System
- Student Management System
- Car Rental System
- Cinema Booking System
- Pet Management System

Simulations

- Traffic Simulation
- Bank Queue Simulation
- Airport Management Simulation
- Hospital Queue Simulation
- Ecosystem Simulation

Utilities

- Personal Expense Tracker
- To-Do List
- File Management Utility
- Password Manager
- Grade Calculator

These are examples only — students are encouraged to propose their own ideas.

5. Project Scope

The project should be sufficiently complex to demonstrate meaningful application of C++ and OOP. A project that consists primarily of a few cin/cout statements and functions will not be considered sufficient.

As a general guideline, students should aim for at least 5 meaningful classes for a basic project, and 6–8 classes for a strong project, with more where justified by the design. The number of classes alone does not determine quality — students must demonstrate meaningful responsibilities per class, and be ready to justify those design choices orally.

6. Required C++ Programming Features

All students must demonstrate appropriate use of basic C++ programming constructs, including:

- Variables and data types (int, double, float, char, bool, string)
- Input and output (cin, cout)
- Operators (arithmetic, relational, logical, assignment)

- Selection (if, else, switch)
- Iteration (for, while, do-while)
- Functions (parameters, return values, decomposition, overloading where appropriate)
- Arrays / STL collections (vector, array, map, set)

7. Required Object-Oriented Programming Features

7.1 Classes and Objects

Students must create and use multiple classes with clear responsibilities.

7.2 Encapsulation

Important data should be appropriately encapsulated using private/protected/public access. Data should not be unnecessarily declared public.

7.3 Constructors

Students should demonstrate appropriate object initialisation using constructors.

7.4 Abstraction

Students should demonstrate abstraction by exposing appropriate interfaces while hiding implementation details, using an abstract class where appropriate.

7.5 Inheritance

Students should demonstrate inheritance where there is a genuine "is-a" relationship. Inheritance should not be added merely to satisfy the marking scheme.

7.6 Polymorphism

Students should demonstrate polymorphism where different derived classes provide different implementations of the same interface.

7.7 Composition / Association

Students should demonstrate appropriate relationships between classes (e.g. a Game composed of Player, Board and Enemy objects).

8. UML Requirements

Students must produce UML diagrams before completing the implementation:

- Use Case Diagram — identifying actors and major system functions.
- Class Diagram — showing classes, attributes, methods, relationships, inheritance and visibility.

9. Advanced C++ Features — Bonus Marks

Students who wish to achieve bonus marks may implement advanced C++ features, used meaningfully rather than simply added for the sake of it:

- Modern memory management (`unique_ptr`, `shared_ptr`)

- Advanced STL (vector, map, set, unordered_map, algorithms, iterators)
- Templates
- Exception handling (try/catch)
- File handling (ifstream/ofstream)
- Operator overloading
- Lambda expressions
- Design patterns, move semantics, const correctness, constexpr, namespaces

10. Minimum Functional Requirements

Depending on the project, students should implement appropriate functions such as add, delete, update, search, display, calculate, sort, process, save, load, start, stop, login, game interaction, score management and reporting.

11. Error Handling and Input Validation

Students must consider invalid input and unexpected situations (e.g. invalid menu selection, invalid number, object not found, duplicate ID, file cannot be opened, empty collection) and respond appropriately rather than terminating unexpectedly.

12. File Storage

Students may store information in memory using vector/map/set, and are encouraged to implement file storage (e.g. data.txt, players.txt, savegame.txt) as an enhancement. File persistence may be considered an advanced/bonus feature depending on the project.

13. Project Development Stages

Stage	Deliverable
1. Project Proposal	Title, problem statement, objectives, target users, main features, proposed classes
2. UML Design / Control Structure	Use case diagram, class diagram, explanation of relationships
3. Prototype	Basic C++ syntax, main menu/loop, basic classes, core functionality
4. OOP Implementation	Encapsulation, abstraction, inheritance, polymorphism, composition
5. Testing	Test cases and identified/fixed bugs
6. Finalisation	Source code, report, UML, test cases, user manual, oral preparation

14. Project Deliverables

14.1 Source Code

Complete, organised C++ source code (e.g. include/, src/, data/, main.cpp, README.md).

14.2 Project Report

Introduction; Problem Statement; Objectives; System Requirements; System Design; UML Diagrams; Class Descriptions; OOP Concepts Used; C++ Features Used; Implementation; Testing; Results; Limitations; Future Improvements; Conclusion; References.

14.3 User Manual

How to compile and run; system requirements; main menu; available functions; input requirements; error messages; how to use advanced features.

14.4 Test Cases

At least 15 test cases (Test ID, Description, Input, Expected Result, Actual Result, Status).

15. Partner-Based Peer Evaluation

Although the project is individual, each student is paired with a partner for evaluation (Student A develops Project A and evaluates Student B's project, and vice versa). The partner is not responsible for developing the project — the partner acts as technical reviewer, user tester and peer evaluator.

The peer evaluation marks assess the quality of the student's review (not the reviewer's opinion of the partner's technical quality), to avoid friendship-based grade inflation. The evaluator should run the program, follow the user manual, test selected functions, inspect the UML diagram, ask the developer questions, and complete the peer evaluation form with evidence-based comments. The lecturer retains final authority over all marks.

16. Assessment — 100 Marks

The assessment is split into two equally-weighted components: (A) Project Development, Documentation and Demonstration, and (B) Oral Presentation. A student who builds a working system, completes the report and demonstrates it competently can secure Part A; Part B is assessed separately and independently through on-the-spot questioning, so strong performance in one part does not guarantee marks in the other.

Part A — Project Development, Documentation & Demonstration (50 Marks)

Criterion	Marks
Basic C++ programming (variables, I/O, selection, loops, functions, STL)	10
OOP concepts (classes/objects, encapsulation, constructors, abstraction, inheritance, polymorphism, composition)	15
Application functionality (core features, usability, algorithms, validation)	8
Software design, UML and code quality (diagrams, structure, readability, testing)	7
Peer evaluation and technical review (quality of review conducted on partner's project)	5
Report completeness and live demonstration of the working application	5
Total	50

Part B — Oral Presentation (50 Marks)

Each student individually answers questions, on the spot, about C++ and OOP concepts and about their own project's code — verifying genuine understanding rather than rehearsed delivery. Ten questions are asked per

student (see Section 17 for examples); each is marked out of 5 based on correctness, clarity, and the ability to relate the concept back to the student's own implementation.

Criterion	Marks
Correctness of conceptual answer (does the student actually understand the concept?)	20
Ability to demonstrate/relate the concept to their own code	20
Clarity of explanation and response to follow-up/modification questions	10
Total	50

Component	Marks
Part A — Project Development, Documentation & Demonstration	50
Part B — Oral Presentation	50
TOTAL	100

Note: This restructures the earlier draft's marking scheme (20/35/20/15/10) by folding it into Part A at half weight, and introduces oral presentation performance as an equally-weighted, independently assessed second half— rather than a small bonus for explaining code during the demo.

17. Oral Presentation — Example Questions

Students should prepare to answer questions such as the following during their individual oral presentation. Each question is designed to require the student to explain a concept and then show or point to where and how they used it in their own project (or explain why they did not need to).

1. How do you define a class in C++, and what is the difference between a class and an object? Show one of your classes and create/instantiate an object of it.
2. What is encapsulation, and why is it useful? Point to a class in your project where you deliberately made data members private or protected, and explain what would go wrong if they were public instead.
3. What is a constructor, and what happens in memory when an object is created? Show one of your constructors and explain each parameter.
4. Explain inheritance in your project. Which classes form a base/derived relationship, and why is that a genuine "is-a" relationship rather than one added just to satisfy the requirement?
5. What is polymorphism, and how have you used it? Show a place in your code where a base-class pointer or reference is used to call a function that behaves differently depending on the actual derived object.
6. What is function overloading (as opposed to overriding)? Give an example from your own code of two functions with the same name but different parameters, and explain why you needed both.
7. What is abstraction, and how does it differ from encapsulation? If you used an abstract class or pure virtual function, show it and explain what it prevents a caller from doing.
8. Explain one relationship (composition or association) between two classes in your project — for example, one class holding objects of another. Why did you model it that way instead of using inheritance?
9. Which STL container(s) did you use (e.g. vector, map, set) and why did you choose that container over a plain array or another container for that particular data?
10. If I asked you right now to add a new derived class (or modify an existing feature), how would you do it in your existing design, and which files/classes would you need to change?

Lecturers may substitute or add questions drawn from the student's own class diagram and source code; the intent is that no two students receive an identical script, and that questions probe the specific design decisions each student made.

18. Bonus Marks

Students may obtain up to an additional 10 bonus marks for demonstrating advanced C++ features (Section 9), correctly implemented and explained during the presentation. The final recorded project score is capped at 100 marks even where bonus marks would otherwise exceed it.

Advanced Feature	Suggested Bonus
File persistence	+2
Exception handling	+1
Smart pointers	+2
Templates	+2
STL algorithms / advanced STL	+1
Operator overloading	+1
Design pattern / advanced architecture	+2
Other significant advanced C++ feature	+1 to +2

19. Academic Integrity

All submitted source code must be the student's own work. Students may use C++ documentation, textbooks, lecture materials, online references and tutorials, but must understand and be able to explain the submitted code. A student who cannot explain substantial portions of their own implementation during the oral presentation may be required to undergo additional verification. Marks will be deducted for plagiarism and late submission.

20. Recommended Project Difficulty

Basic: Quiz Game, Calculator, To-Do List, Simple Inventory System. Intermediate: Blackjack, Battleship, Library/Restaurant/Student Management System. Advanced: RPG, Tower Defense, Chess, Banking Simulation, Hospital Management System. Students are not required to select an advanced project to obtain a high mark — assessment is based on the quality and appropriateness of the implementation, and on the student's understanding demonstrated at the presentation.

21. Final Submission Checklist

- Source code, organised and complete
- Video demo link in Youtube (less than 10 minutes with explanation)
- Project report (all required sections)
- Use case diagram and class diagram
- User manual
- At least 15 test cases
- Partner identified and peer evaluation completed

- Student has rehearsed explaining every class and OOP concept used, in preparation for the oral presentation

22. Final Project Philosophy

The purpose of this project is not to see who can build the most complicated application. Students should demonstrate that they can take a problem of their choice and transform it into a structured C++ application using appropriate object-oriented principles: Understand → Design → Implement → Test → Explain.

Every student should be able to answer, in their own words during the oral presentation: "Why did you design your program this way?" This is weighted equally with building the system itself, because it is the clearest evidence that the learning objectives — not just a working program — have been achieved.